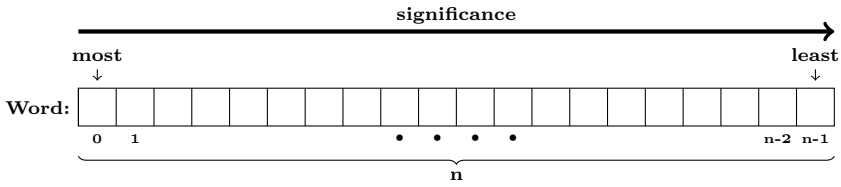
Word Class Documentation

Source File: Word.h
Namespace: cae
Class Header: `class Word : public Object`

Overview

The *Word* class simulates a comparable binary word [most significant bit to least significant bit is read from right to left] that can prohibit resizing and be linked.



Constructors

- `Word()` (default constructor)
  - **Purpose**: Constructs an empty word with an adjustable length.
- `Word(const Word& obj)` (copy constructor)
  - **Purpose**: Performs a deep copy of *obj*.
  - **Parameter(s)**:
    - *obj*: Constant *Word* reference object.
- `Word(size_t sz)`
  - **Purpose**: Constructs a *sz*-bit word of zeroes with an adjustable length.
  - **Parameter(s)**:
    - *sz*: Unsigned integer.
- `Word(string str)`
  - **Purpose**: Converts a binary string *str* into word with an adjustable length if *str* is a binary string; otherwise, construct an empty word.
  - **Parameter(s)**:
    - *str*: String.

Destructor

- `~Word()` [virtual]
  - **Purpose**: Unlinks the word attachment.

Assignment Operator

- `operator=(const Word& rhs)`
  - **Purpose**: Performs a deep copy of *rhs* only if the word has an adjustable length
  - **Parameter(s)**:
    - *rhs*: Constant *Word* reference object.
  - **Return**: `*this`.

Member Functions

- `size() const`
  - **Purpose**: Returns the length of the word.
  - **Return**: Unsigned integer.

- `get(size_t idx) const`
  - **Purpose**: Returns a bit from the word at *idx* if *idx* is valid; otherwise, false.
  - **Parameter(s)**:
    - *idx*: Unsigned integer.
  - **Return**: Boolean.
- `set(size_t idx,bool bit)`
  - **Purpose**: Sets word bit at *idx*.
  - **Parameter(s)**:
    - *idx*: Unsigned integer.
    - *bit*: Boolean.
- `set(bool value)`
  - **Purpose**: Sets all bits of the word to *value*
  - **Parameter(s)**:
    - *value*: Boolean.
- `fix()`
  - **Purpose**: Makes the word's length unadjustable if it is not empty.
- `fixed() const`
  - **Purpose**: Checks if the length of the word is adjustable.
  - **Return**: Boolean.
- `empty() const`
  - **Purpose**: Checks if the word is empty.
  - **Return**: Boolean.
- `extended() const`
  - **Purpose**: Checks if the word has an attachment.
  - **Return**: Boolean.
- `appended() const`
  - **Purpose**: Checks if the word is an attachment.
  - **Return**: Boolean.
- `bounded() const`
  - **Purpose**: Checks if the word has or is an attachment.
  - **Return**: Boolean.
- `unbind()`
  - **Purpose**: Removes attachment from the word and the word as an attachment.
- `join(Word& obj)`
  - **Purpose**: Attaches *obj* to the end of the word if both are not empty, the word has no attachment, and *obj* is not bounded.
  - **Parameter(s)**:
    - *obj*: *Word* reference object.
- `toString() const` [private overridden]
  - **Purpose**: Represents *Word* object as a binary string if not empty; otherwise, nil
  - **Return**: String.

Non-Member Functions

- `subword(const Word& obj,size_t i,size_j)`
  - **Purpose**: Retrieves a subword from *obj* between indices *i* and *j*, inclusively, if *i* and *j* are both valid indices; otherwise, it retrieves an empty word.
  - **Parameter(s)**:
    - *obj*: Constant *Word* reference.
    - *i*: Unsigned integer.
    - *j*: Unsigned integer.
  - **Return**: *Word* object.
- `transfer(Word& lhs,const Word& rhs)`
  - **Purpose**: Transfers the content of *rhs* to *lhs* if they are different *Word* objects with the same length.
  - **Parameter(s)**:
    - *lhs*: *Word* reference.
    - *rhs*: Constant *Word* reference.
- `value(const Word& itm)`
  - **Purpose**: Provides conversion of a *Word* object to unsigned integer if the value of *itm* does not exceed $2^{32} - 1$; otherwise, zero.
  - **Parameter(s)**:
    - *itm*: Constant *Word* reference object.
  - **Return**: Unsigned integer.
- `operator>>(istream& i,Word& obj)`
  - **Purpose**: Reads in a word if the input is a binary string and the word is adjustable; otherwise, makes the word nil
  - **Parameter(s)**:
    - *i*: Istream reference object.
    - *rhs*: *Word* reference object.
  - **Return**: Boolean.
- `operator==(const Word& lhs,const Word& rhs)`
  - **Purpose**: Check if *lhs* and *rhs* have equal values.
  - **Parameter(s)**:
    - *lhs*: Constant *Word* reference object.
    - *rhs*: Constant *Word* reference object.
  - **Return**: Boolean.
- `operator!=(const Word& lhs,const Word& rhs)`
  - **Purpose**: Check if *lhs* and *rhs* have different values.
  - **Parameter(s)**:
    - *lhs*: Constant *Word* reference object.
    - *rhs*: Constant *Word* reference object.
  - **Return**: Boolean.